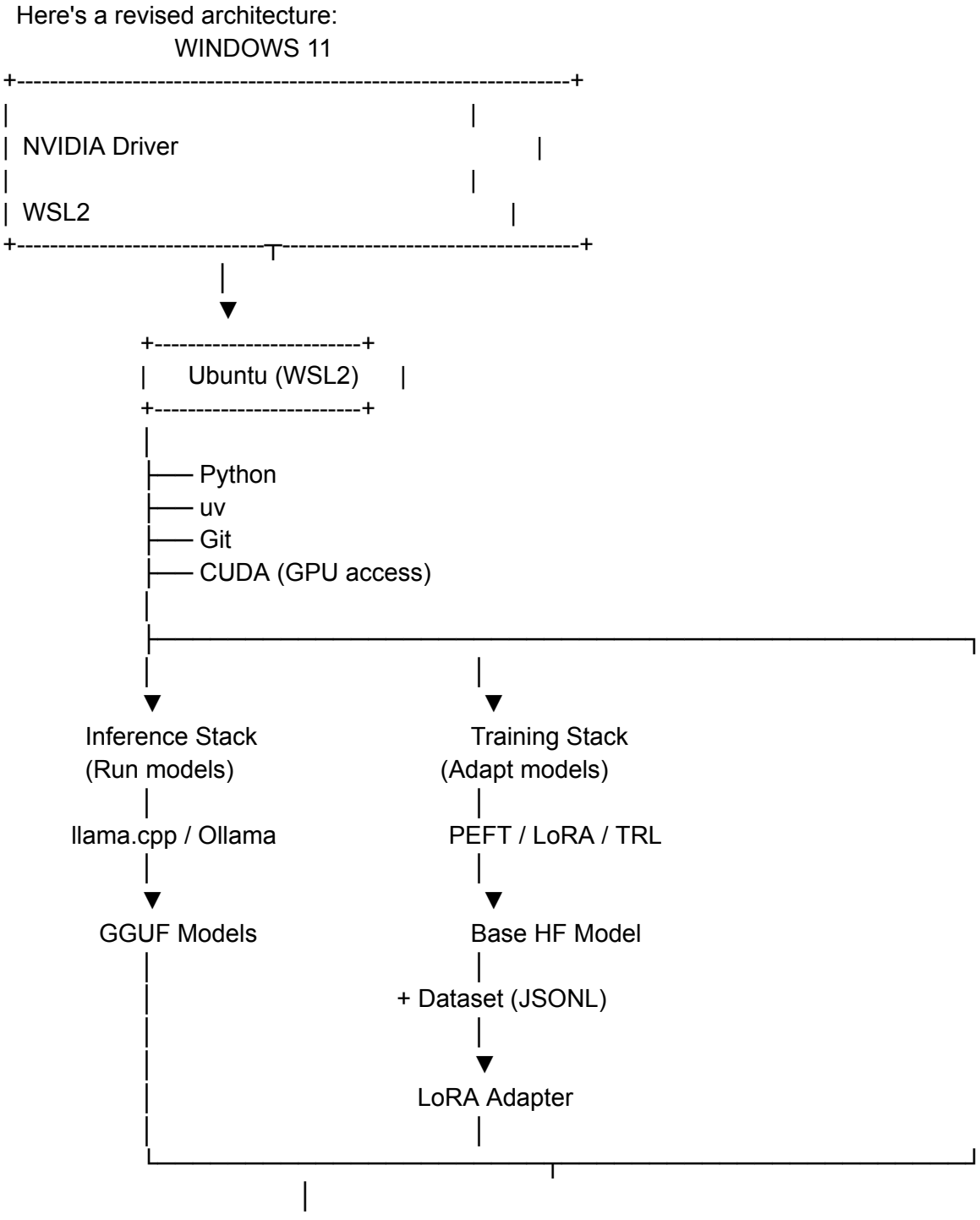
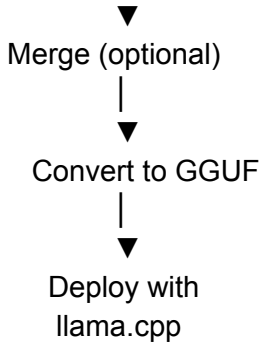
AI LAB OFFLINE/TUNNING

Your AI Lab Architecture

Here's a revised architecture:

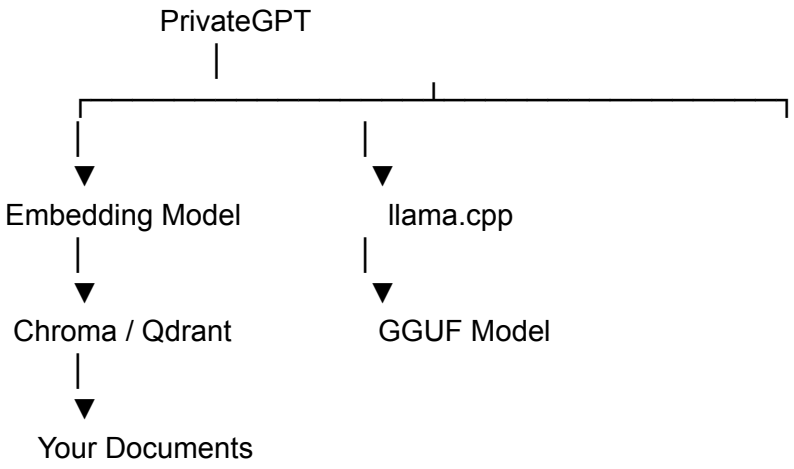




Where PrivateGPT fits

PrivateGPT is **not** part of the training stack.

It's an **application** that uses the inference stack.



Notice that:

- **PrivateGPT never trains the model.**
- It simply retrieves relevant document chunks and asks the LLM to answer based on them.

Your complete ecosystem

Eventually, your machine will look something like this:

Ubuntu WSL
│
├── AI Runtime

```
│   ├── llama.cpp
│   ├── Ollama (optional)
│   └── CUDA
│
├── AI Applications
│   ├── PrivateGPT
│   ├── Open WebUI (optional)
│   └── CLI tools
│
├── Models
│   ├── Base GGUF
│   ├── LoRA adapters
│   └── Merged GGUF
│
├── Training
│   ├── PEFT
│   ├── TRL
│   ├── Transformers
│   └── Datasets
│
└── Data
    ├── Documents
    ├── Chroma DB
    └── Embeddings
```

I like this view because it separates **runtime**, **applications**, **training**, **models**, and **data**. Once you see those as independent pieces, the whole local AI ecosystem becomes much easier to understand.

One more important distinction

You'll notice I separated **Base HF Model** and **GGUF Models**. That's because:

- **Training** is normally done on the original Hugging Face model format (PyTorch/Transformers).
- **Inference with `llama.cpp`** uses **GGUF** models.

So a typical lifecycle is:

```
Base Hugging Face model
        │
        ▼
LoRA training
        │
        ▼
```

LoRA adapter



(Optional) merge



Convert to GGUF



Run with llama.cpp

That distinction is one of the key concepts to understand early—it explains why there are different model formats and why you don't usually fine-tune a GGUF model directly.

Project Road Map 2026

Phase 1 – Build your AI workstation (1–2 days)

Goal: Have a stable environment where you can run local models.

Windows 11



NVIDIA Driver



WSL2



Ubuntu

Install:

Ubuntu

├── Git

├── Python

├── uv

├── build-essential

├── CUDA support

└── VS Code

At the end of this phase you should be able to run:

nvidia-smi

python --version

uv --version

git --version

Phase 2 – Learn inference

Forget PrivateGPT for now.

Just learn to run a model.

Ubuntu

│

└── llama.cpp

    │

    ▼

Llama 3.1 8B

Try prompts like

Explain recursion.

Write a Python function.

Summarise this paragraph.

The goal isn't to build anything—it's to understand how a local LLM behaves.

Phase 3 – Learn RAG

Only after you're comfortable with inference.

PDF

↓

Embedding

↓

Vector Database

↓

LLM

↓

Answer

This is where PrivateGPT comes in.

You'll learn:

embeddings

chunking

retrieval

vector databases

These concepts are used in almost every modern AI application.

Phase 4 – Build datasets

This is where many beginners underestimate the work.

Instead of training immediately, learn to create good examples.

Example:

Question

↓

Expected Answer

Eventually you'll have hundreds or thousands of these.

Phase 5 – LoRA

Only now.

Dataset

↓

Base Model

↓

LoRA

↓

Adapter

Notice something important.

You don't create another 5 GB model.

You create a small adapter.

Base model

5 GB

+

LoRA adapter

100–500 MB

↓

Specialised assistant

That's why LoRA is so popular.

Eventually

Base Model

+

Multiple LoRAs

↓

Medical Assistant

↓

Programming Assistant

↓

Engineering Assistant

↓

Finance Assistant

One base model can support multiple specialised adapters.

Your folder structure

I'd build it like this.

```
~/AI
│
├── models
│   ├── base
│   ├── lora
│   └── merged
│
├── datasets
│   ├── raw
│   ├── processed
│   └── jsonl
│
├── privateGPT
│
├── llama.cpp
│
```

├── notebooks

│

└── experiments

You'll appreciate this separation after a few projects.

Learning roadmap

Week 1

———————————

✓ Linux

✓ Git

✓ Python

✓ WSL

✓ CUDA

↓

Week 2

———————————

✓ llama.cpp

✓ GGUF

✓ Prompting

↓

Week 3

———————

✓ Embeddings

✓ Chunks

✓ Chroma

✓ PrivateGPT

↓

Week 4

———————

✓ Dataset creation

✓ JSONL

✓ Instruction tuning

↓

Week 5

———————

✓ LoRA

✓ PEFT

✓ Evaluation

↓

Week 6

✓ Deploy

✓ Optimize

✓ Benchmark

The concepts worth learning

Rather than memorising commands, focus on understanding these ideas:

Concept	Why it matters
Tokens	Explains context limits and generation costs.
Context window	Determines how much text the model can consider at once.
Quantisation (Q4, Q5, Q8)	Explains the trade-off between memory use, speed, and quality.
GGUF	The model format commonly used with llama.cpp.
Embeddings	Power semantic search over your documents.
Vector database	Stores embeddings for fast retrieval.

RAG Combines retrieval with generation for document-based answers.

LoRA Efficient way to specialise a model without retraining everything.

These concepts transfer across tools and models.

My suggestion for your first project

I'd make your first goal intentionally simple:

Windows

WSL Ubuntu

llama.cpp (CUDA)

Llama 3.1 8B GGUF

Ask questions locally

Don't install PrivateGPT yet.

Once you can reliably:

run the model on your GPU,

understand what the context window is,

see how changing prompts affects responses,

then add PrivateGPT and your documents. By learning one layer at a time, you'll have a much easier time understanding and troubleshooting your setup, and you'll build a solid foundation for LoRA fine-tuning later.

First things first

Checking NVDIA drivers

Nvidia-smi

wsl – install
wsl.exe --isntall --no-distribution
wsl --status
wsl -l -v

Install ubuntu from MS store

Done

Change Ubuntu hostname

sudo nano /etc/wsl.conf

[network]
hostname = llama-host

wsl – shutdown

```
sudo apt update
sudo apt upgrade -y

sudo apt update
sudo apt install -y git build-essential curl wget unzip pkg-config //Install development tools

pip3 --version

curl -LsSf https://astral.sh/uv/install.sh | sh
uv --version

Create folders

mkdir -p ~/AI
mkdir -p ~/AI/{models,datasets,training,privateGPT,llama.cpp,experiments}

nvidia-smi --query-gpu=name,memory.total --format=csv //check pgu name memory

cd ~/AI/privateGPT
python3.11 --version
sudo apt install python3.11 or uv python install 3.11
uv venv --python 3.11
uv python list
source .venv/bin/activate
```

Clone/Git privateGPT clone repository

```
cd ~/AI/privateGPT
git clone https://github.com/zylon-ai/private-gpt.git .
```

Setting up settings.yaml

For your PrivateGPT `settings.yaml`, the relevant part should look like:

```
models:
  - type: llm
    name: llama3.1:8b
    mode: openai
    provider: ollama
    enabled: true
    alias: llama3.1:8b
```

```
    context_window: 4096
    prompt_style: chat
    tokenizer_mode: default
    api_type: chat_completions
    sampling_params:
      seed: 0
      temperature: 0.1
      max_new_tokens: 512

  - type: embedding
    name: nomic-embed-text
    mode: openai
    provider: ollama
    enabled: true
    context_window: 8192
```

cd ~/AI/privateGPT
source .venv/bin/activate
export OPENAI_API_BASE=http://127.0.0.1:11434/v1
PGPT_SETTINGS_FOLDER=./settings uv run private-gpt serve

Setting up settings-model.yaml

```
llm:
  auto_discover_models: false
  default_model: llama3.1:8b
embedding:
  auto_discover_models: false
  default_model: ''
models:
  - type: llm
    name: llama3.1:8b
    mode: openai
    provider: ollama
    enabled: true
    alias: llama3.1:8b
    context_window: 128000
    prompt_style: chat
    tokenizer_mode: default
```

```
  support_image: 1
  support_audio: 0
  api_type: chat_completions
  sampling_params:
    seed: 0
    temperature: 0.1
    max_new_tokens: 1024
    min_p: 0.1
    top_p: 0.7
    top_k: 50
    repetition_penalty: 1.2
    presence_penalty: 0.0
    frequency_penalty: 0.0
  reasoning_sampling_params:
    seed: 0
    temperature: 0.1
    max_new_tokens: 1024
    min_p: 0.1
    top_p: 0.7
    top_k: 50
    repetition_penalty: 1.2
    presence_penalty: 0.0
    frequency_penalty: 0.0
  tags: []
```

Then install dependencies

```
uv sync --extra llms-llamacpp // project does not support extras
grep -A30 "\[project.optional-dependencies\]" pyproject.toml //check if pyproject.toml has private-gpt extras
grep -A100 "\[project.optional-dependencies\]" pyproject.toml | grep -i -A5 -B5 llama //check if privateGPT exposes llm-llamacpp extras
grep -i -n "llama" pyproject.toml //fails correctly

python -c "import llama_cpp; print(llama_cpp.__version__)"

nvcc --version
```

Good, this tells us exactly where we are:

ModuleNotFoundError: No module named 'llama_cpp'

*Your PrivateGPT code supports llama.cpp, but the **llama.cpp Python backend is not installed yet**.*

*Since you have an RTX 3070 Ti and want GPU acceleration, **do not install the normal PyPI package**:*

uv pip install llama-cpp-python

because that commonly installs a CPU-only build.

We need to install it with CUDA enabled.

Installing CUDA toolkit

nvcc --version //if returns nothing

cd ~
wget
https://developer.download.nvidia.com/compute/cuda/repos/wsl-ubuntu/x86_64/cuda-keyring_1.1-1_all.deb

sudo dpkg -i cuda-keyring_1.1-1_all.deb

sudo apt-get update

sudo apt-get -y install cuda-toolkit-13-3

Check if everything installed fine:

nvcc --version
ls /usr/local/
dpkg -l | grep cuda-toolkit

Add CUDA to your environment:

echo 'export PATH=/usr/local/cuda/bin:$PATH' >> ~/.bashrc
echo 'export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH' >> ~/.bashrc

cd ~/AI/privateGPT
source .venv/bin/activate

CMAKE_ARGS="-DGGML_CUDA=on" uv pip install llama-cpp-python --force-reinstall
--no-cache-dir // force isntall CUDA-enabled llama.cpp backend

```
python -c "from llama_cpp import llama_supports_gpu_offload;
print(llama_supports_gpu_offload())" // checks
python -c "import llama_cpp; print(llama_cpp.__version__)" //checks


ls -lh ~/AI/privateGPT/models/ //make sure GGUG exists
```

Current Stack Diagram:

RTX 3070 Ti
↓
CUDA 13.3 Toolkit
↓
llama-cpp-python 0.3.34 (CUDA)
↓
PrivateGPT
↓
Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf

Download/Run Model into project

```
mkdir -p ~/AI/privateGPT/models // creates model folder

cd ~/AI/privateGPT
source .venv/bin/activate
cd models
uv pip install huggingface_hub

cd ~/AI/privateGPT/models

sudo apt-get install zstd

curl -fsSL https://ollama.com/install.sh | sh //download model / brain into models folder

ollama pull llama3.1:8b
ollama list
ollama ps
ollama run llama3.1:8b
ollama ps
```

That's excellent news.

Your output confirms:

PROCESSOR 100% GPU

Ollama is using your RTX 3070 Ti correctly. 🎉

This means:

- ✅ NVIDIA driver is working
- ✅ WSL GPU passthrough is working
- ✅ Ollama is using the GPU
- ✅ Your CUDA setup is good enough for inference

Why does it still feel slow?

A few possibilities:

1. **First prompt is always slower**
    - Ollama has to load the 5.3 GB model into VRAM.
    - Subsequent prompts are usually much faster while the model stays loaded.
2. **You're running the 8B model**
    - On an RTX 3070 Ti (8 GB VRAM), this is a good fit, but it's not going to generate at hundreds of tokens per second.
    - Typical speeds are often in the **20–50 tokens/sec** range depending on the quantization and prompt.
3. **Long prompts or long outputs**
    - The more context or output requested, the longer it takes.

Install dependencies panda, magic, htmltext, htmltext2, celery, httpx2

```
cd ~/AI/privateGPT
source .venv/bin/activate

# System dependency for python-magic
sudo apt update
```

```
sudo apt install -y libmagic1

# Add Python dependencies
uv add pandas python-magic html-text html2text celery httpx httpx2

# Sync environment
uv sync
```

Final check

```
python - <<'PY'
import pandas
import magic
import html_text
import html2text
import celery
import httpx

print("pandas OK:", pandas.__version__)
print("magic OK")
print("html-text OK")
print("html2text OK")
print("celery OK:", celery.__version__)
print("httpx OK:", httpx.__version__)
print("httpx2 OK:", httpx.__version__)
print("All dependencies OK")
PY

git remote -v
git branch --show-current
git log -1 --oneline

uv sync --inexact --extra embedding-openai-compatible

python -c "import llama_index.embeddings.openai_like; print('OK')"

uv sync --inexact --extra llm-openai // The OpenAI provider dependencies are installed:

OPENAI_API_BASE=http://127.0.0.1:11434/v1 uv run python scripts/auto_discover_models.py
--out settings-model.yaml // connect Ollama/brain into privateGPT UI

Expect

Wrote settings-model.yaml with 1 LLM models and 0 embedding models.
```

ls -1 *.yaml //check settings folder
mkdir -p settings //create settings folder
mv settings.yaml settings/
mv settings-model.yaml settings/

ls -lah settings/

cd ~/AI/privateGPT
source .venv/bin/activate

uv sync --inexact --extra llm-openai-compatible
python -c "import llama_index.llms.openai_like; print('openai_like OK')"

ollama pull nomic-embed-text

(privateGPT):~/AI/privateGPT/settings$ ollama list
NAME ID SIZE MODIFIED
nomic-embed-text:latest 0a109f422b47 274 MB 11 seconds ago
llama3.1:8b 46e0c10c039e 4.9 GB 16 hours ago

source .venv/bin/activate
uv sync --inexact --extra vectorstore-qdrant
python -c "import qdrant_client; print('Qdrant OK')"

✅ Git repo
✅ Python 3.11 venv
✅ PrivateGPT starts
✅ CUDA toolkit installed
✅ llama-cpp CUDA built
✅ Ollama GPU working
✅ PrivateGPT connects to Ollama
✅ LLM registered
➡️ fixing embedding schema now

Don't I need poetry?

No. **You do not need Poetry** for the version you're using.

Your repository is:

zylon-ai/private-gpt
main

and you've already been using `uv` successfully:

- ✅ `uv venv`
- ✅ `uv sync`
- ✅ `uv run private-gpt serve`
- ✅ `uv pip install ...`

This project has moved to **uv** as its recommended package manager and workflow. Installing Poetry would just add another dependency manager and could create conflicting virtual environments.

FINALLY RUN PRIVATE GPT

cd ~/AI/privateGPT
source .venv/bin/activate
export OPENAI_API_BASE=http://127.0.0.1:11434/v1
PGPT_SETTINGS_FOLDER=./settings uv run private-gpt serve

12:19:19.856 [INFO] [MainThread] uvicorn.error - Application startup complete.

12:19:19.856 [INFO] [MainThread] uvicorn.error - Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)

🎉 **Yes — PrivateGPT is now running successfully.**

These lines confirm it:

Application startup complete.
Uvicorn running on http://0.0.0.0:8080

So your current stack is successfully starting:

- ✅ PrivateGPT
- ✅ Llama 3.1 8B through Ollama
- ✅ Embeddings
- ✅ Qdrant vector store
- ✅ SQLite migrations
- ✅ Web server on port `8080`

Open the UI

http://localhost:8080/docs/

You should get the Swagger/OpenAPI interface if the API is available.

Also try:

http://localhost:8080/ui/